

串口通讯协议应用指引

简介

为便于主控端系统通过串口方式对 TFTLCD 屏上的 UI 控件进行显示属性的修改与控制，华芯微特上位机软件 UI_Creator 针对 LVGL 上位机平台和 μ GUI 上位机平台提供了对应的串口协议方案供用户使用，用户只需调用相关接口函数即可生成所需的数据帧进行串口通讯，进而实现串口屏应用，同时上位机软件也支持串口屏仿真模拟，可实时模拟接收到相关串口数据指令后的图形显示效果，大大提升了前期开发验证的便捷性。

目录

简介	1
一、 基于 LVGL 上位机平台的串口通讯协议	3
二、 基于 μ GUI 上位机平台的串口通讯协议	11
三、 串口协议指令生成工具	20
四、 虚拟串口仿真调试	21

一、 基于 LVGL 上位机平台的串口通讯协议

LVGL 上位机软件工程串口数据解析模块默认使用 UART0，模式为 8N1，波特率为 115200，以上配置可通过程序进行修改。主控端到屏端的通讯数据指令分为两种，分别为设置（setter 控制帧）与查询（getter 控制帧）。setter 控制帧用于对指定控件（由上位机生成）的属性进行修改，getter 控制帧用于获取指定控件（由上位机生成）的当前属性及状态。与之相对应的，屏端依据主控端发送的控制帧（setter 或 getter）类别回应相应的应答帧，以上帧数据结构的定义如下：

（1）setter 控制帧(主控→屏):

字节数	说明	层次
2	帧头(FA 80)	链路层
2	数据长度	
1	MsgId(用户自定义)	数据层
1	指令数	
2	操作对象 1 的 ID	
2	属性码 1	
n	参数 1	
2	操作对象 2 的 ID	
2	属性码 2	
n	参数 2	
n		链路层
2	CRC16	

setter 应答帧(屏→主控):

字节数	说明	层次
2	帧头(FB 80)	链路层
2	数据长度	
1	MsgId	数据层
1	执行结果数	
1	执行结果 1	
1	执行结果 2	
1		
1	执行结果 n	
2	CRC16	链路层

（2）getter 控制帧(主控→屏):

字节数	说明	层次
2	帧头(FA 81)	链路层
2	数据长度	
1	MsgId(用户自定义)	数据层
1	指令数	
2	操作对象 1 的 ID	
2	属性码 1	
2	操作对象 2 的 ID	
2	属性码 2	
2	CRC16	链路层

getter 应答帧(屏→主控):

字节数	说明	层次
2	帧头(FB 81)	链路层
2	数据长度	
1	MsgId	数据层
1	查询结果数	
n	结果 1 数据	
n	结果 2 数据	
2	CRC16	链路层

控件的属性控制与 EZOC 低代码平台的功能相同，具体控件的可控制属性可参照上位机 EZOC 低代码平台文档-华芯微特 EZOC 应用笔记，数据帧中的属性码结构定义及位域描述如下：

15	14	13	12	11	10	9	8
参数类型				EZOC 属性码高 4 位			
7	6	5	4	3	2	1	0
EZOC 属性码低 8 位							

位域	名称	描述
15:12	参数类型	0000: 32 位整数 0001: 字符串
11:0	EZOC 属性码	

其中，对于数据参数的描述字节结构定义如下：

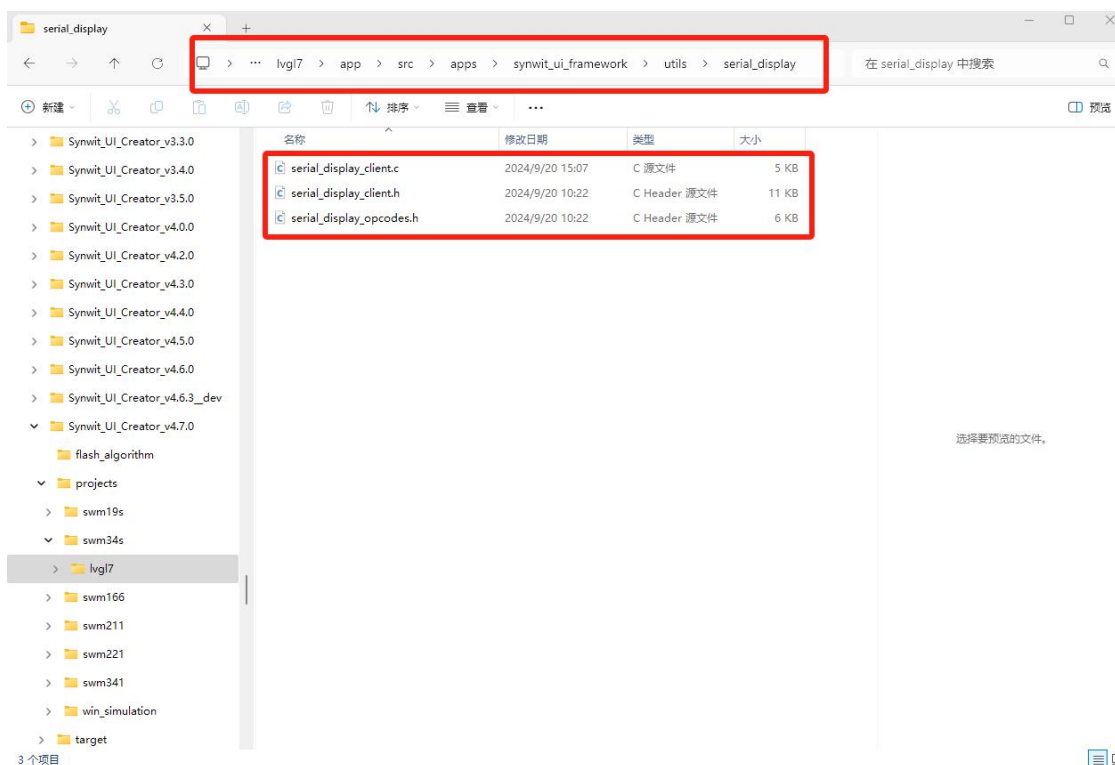
(1) 字符串数据定义：

字节数	说明
2	字符串长度(不包括结尾'\0')
n	UTF8 编码字符串
1	字符串结束标记'\0'

(2) 32 位数值数据定义：

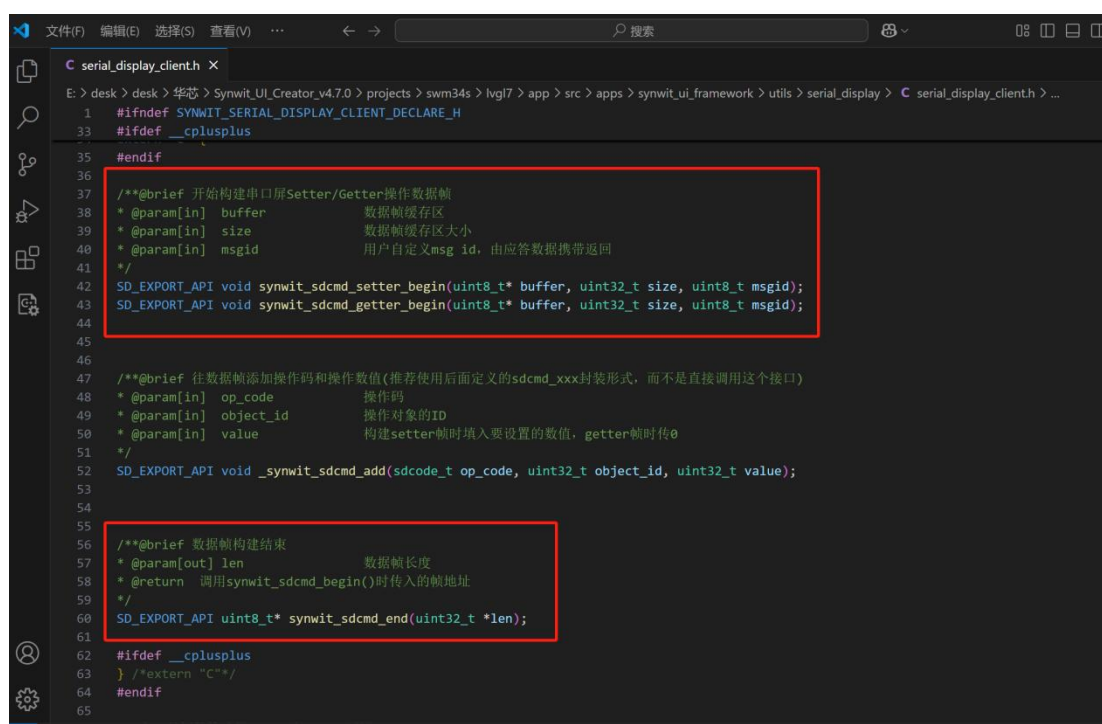
字节数	说明
4	32 位数值

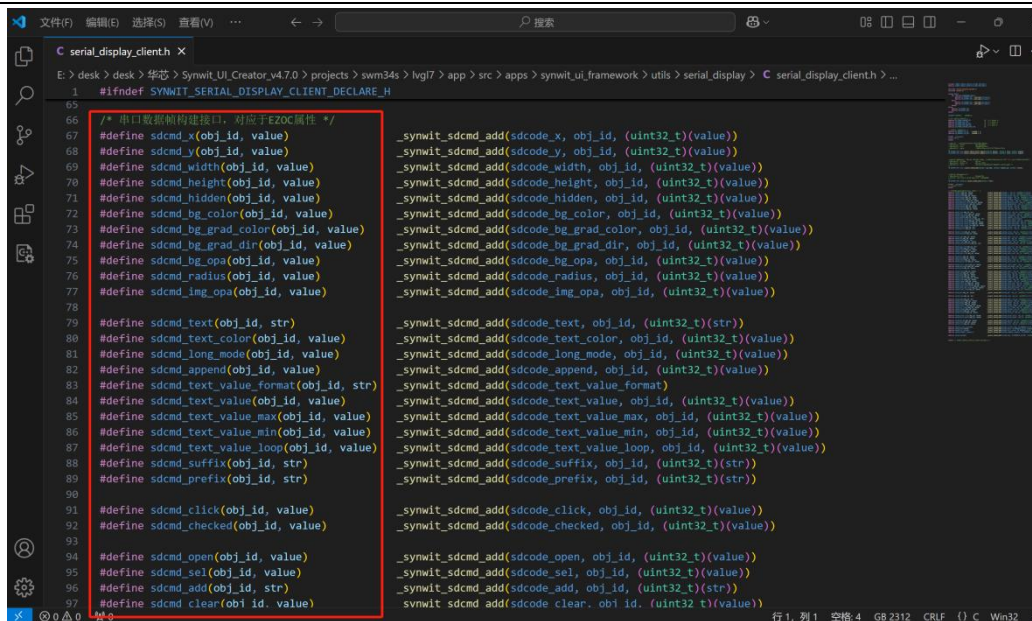
主控端系统程序需通过调用相关函数来进行控制帧数据包生成。打开 UI_Creator 上位机 swm34s 工程文件夹，在..\swm34s\lvg17\app\src\apps\synwit_ui_framework\utils\serial_display 路径中将程序源码文件拷贝到主控端工程中。



拷贝移植成功后，在 `serial_display_client.h` 中可以看到用于串口数据帧构建的功能接口函数声明，构建步骤围绕 API 接口分为：传入数据缓存区开始构建->数据帧添加控件操作命令->结束构建返回数据帧地址。

使用 `synwit_sdcmnd_xxxx_begin` 构建起始函数将开辟的数据帧缓存区传入用于数据装载，`msgid` 为用户提供了自定义信息标识传入，在构建的数据帧及应答帧中特定数据位体现，方便用户进行帧甄别；启动构建后即可通过往数据帧中添加控件操作码、ID 及数值，实现对对应控件属性的设置或获取，鉴于控件属性的种类多样化，推荐用户使用封装好的针对特定属性（对应 EZOC 开放的功能控制属性）的 `sdcmnd_xxx` 函数接口进行使用，数据帧中支持多个不同控件、不同属性的命令载入，通过连续调用不同 `sdcmnd_xxx` 接口实现；完成构建后需调用 `synwit_sdcmnd_end` 函数接口结束数据帧构建，传入数据长度变量指针用于获取最终的数据帧长度。完成以上步骤后，即可获得构建的具有特定操作功能的串口数据帧及其长度，借此便可以使用串口进行数据发送，进而操作从机端使用 UI_Creator 上位机创建的控件的属性。





当需要对从机端上位机生成的控件的属性进行控制更改时，结合上述操作步骤，构建 setter 数据帧搭建示例：

1. `uint8_t buf[2048];`
2. `uint32_t len;`
3. `//开始构筑 setter 数据帧，getter 数据帧使用 synwit_sdcmd_getter_begin`
4. `synwit_sdcmd_setter_begin(buf, sizeof(buf), 1);`
5. `//添加需要的属性控制`
6. `sdcm_level(2, 10);`
7. `sdcm_value(3, 20);`
8. `//完成构筑`
9. `synwit_sdcmd_end(&len);`
10. `//通过串口将数据帧发送`
11. `uart_write_buff(buf, len);`

(1) 首先，程序中开辟出串口数据缓存数组 `buf[2048]` 来存储数据，缓存区大小结合数据实际应用需求设置即可，需声明一个长度变量用于获取数据帧长度。

(2) 将缓存区地址传入，设置帧自定义 `msgid` 为 1，启动构建。

(3) 随后添加控件操作命令，这里以可以控制分级图像级数的 `sdcm_level` 接口及控制进度条或滑块数值的 `sdcm_value` 接口为例：传入上位机分配的控件 ID 为 2 的分级图像，设置级数为 10；传入上位机分配的控件 ID 为 3 的滑块（进度条），设置当前数值为 20。

(4) 结束构建，获得数据帧长度。

(5) 使用串口发送函数（该函数为主控程序所使用的串口发送函数）将数据发出。

通过编写、执行以上代码段，便可在主控端构建并发送出对应的串口数据包。主控端串口通过 USB 与电脑连接，使用串口助手接收查看发出的数据帧，结合前面所描述的帧结构定义，对数据帧的解构如下：



将以上生成的数据帧通过串口发送至从机屏端，可观察到从机屏端显示的控件 ID 为 2 的分级图像的级数跳转为 10，控件 ID 为 3 的进度条的数值跳变为 20，从机屏端将会返回包含执行结果的应答帧到主机端，使用电脑串口助手查看应答数据如下，表示为所发送的设置命令均已执行成功。



当需要获取相关控件的实时状态属性时，则需要进行 getter 控制帧构建，这里使用 synwit_sdccmd_getter_begin 接口进行数据构筑，构筑流程与 setter 控制帧构筑流程大同小异。值得注意的是，构筑 getter 控制帧时，sdccmd 函数的传参仅需考虑控件 ID 传入，而数据参数传参可为任意值，参考代码如下：


```

1.  uint8_t buf[2048];
2.  uint32_t len;
3.  //开始构筑 getter 数据帧
4.  synwit_sdcmd_getter_begin(buf, sizeof(buf),1);
5.  //添加需要查询的属性
6.  sdcmd_value(8,0);
7.  //完成构筑
8.  synwit_sdcmd_end(&len);
9.  //通过串口将数据帧发送
10. uart_write_buff(buf, len);

```

通过编写、执行以上代码段，便可在主控端构建并发送出对应的串口数据包，所实现的功能为发送构筑的 **getter** 控制帧，查询控件 ID 为 8 的进度条或滑块控件的当前数值。主控端串口通过 **USB** 与电脑连接，使用串口助手接收查看发出的数据帧，结合前面所描述的帧结构定义，对数据帧的解构如下：



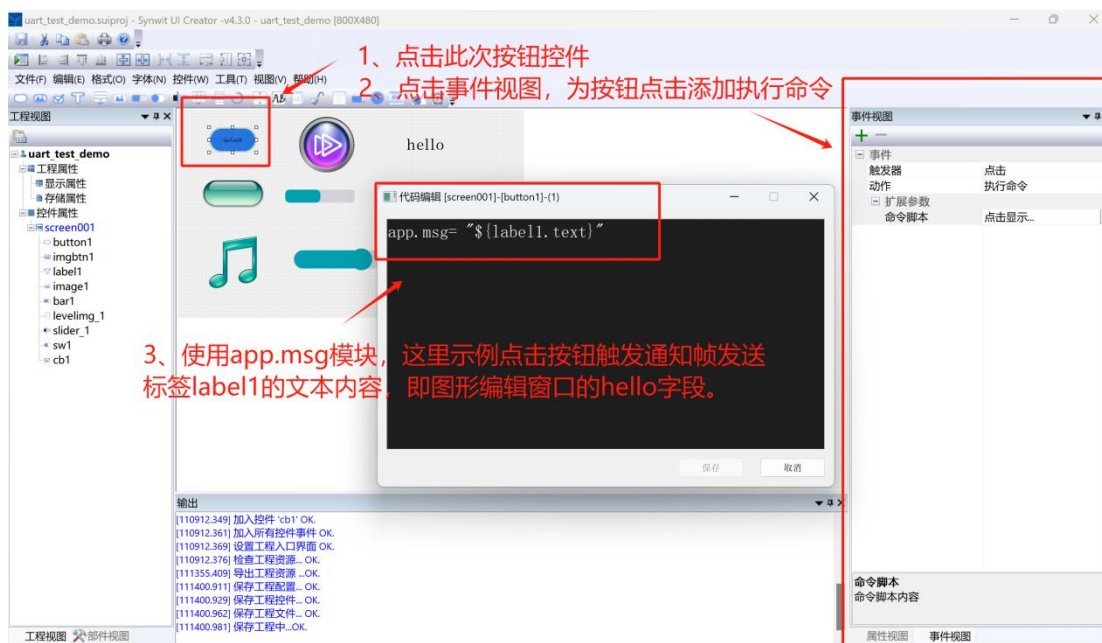
将生成的数据包由主机端通过串口发送至从机屏端，即可获取到相关的控件信息，屏端返回包含执行结果的应答帧如下，其表示为查询控件 ID 为 8 的进度条或滑块控件的当前数值为 100。



华芯微特串口协议方案支持用户自定义通知帧功能，用于屏端交互时通过指定事件触发自定义通知帧数据发送，通知帧数据帧结构定义如下：

字节数	说明	层次
2	帧头(FB 83)	链路层
2	数据长度	
n	用户自定义数据	数据层
2	CRC16	链路层

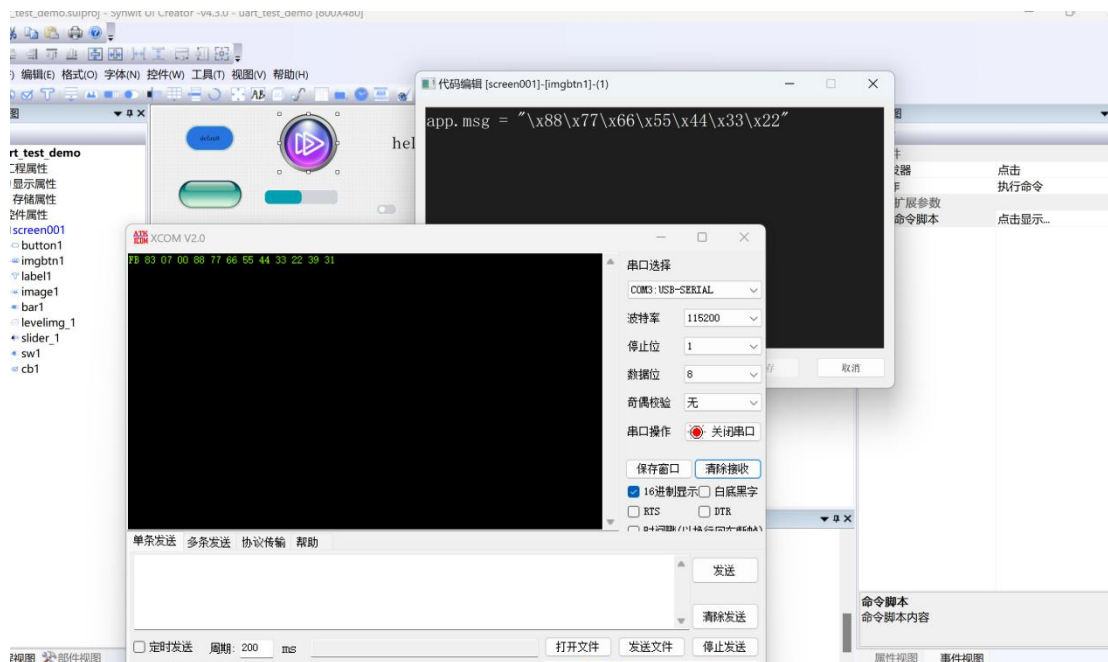
帧数据的自定义数据需结合上位机 EZOC 低代码平台的 app.msg 模块使用，详参上位机 EZOC 低代码平台文档-华芯微特 EZOC 应用笔记，示例如下：



如上述示例所示，通过为按钮控件添加点击事件触发通知帧发送功能，保存后，点击按钮控件，屏端将通过窗口发送通知帧，使用串口助手查看数据包如下：



以此类推，构造十六进制数据时，示例如下：



针对诸如检验码错误、对错误控件进行操作等情况，协议使用通信出错帧形式反馈到主控，目前错误码统一使用 0，数据帧具体结构定义如下：

字节数	说明	层次
2	帧头(FB FF)	链路层
1	错误码(目前为 0)	
2	CRC16	

二、 基于μGUI 上位机平台的串口通讯协议

μGUI 上位机平台支持通过平台拟定的串口通讯协议实现对控件的样式属性进行操作控制，使用方式与 LVGL 平台类似。主控端到串口屏端的协议数据指令统称为 **setter** 控制帧，通过构建不同的 **setter** 控制帧数据包，即可实现对串口屏端上位机生成的指定控件的特定属性样式进行更改，与之相对应的，串口屏会就执行结果返回应答数据包 **setter_ACK**，将执行结果信息反馈到主控。**setter** 控制帧和 **setter** 应答帧帧数据结构定义如下所示：

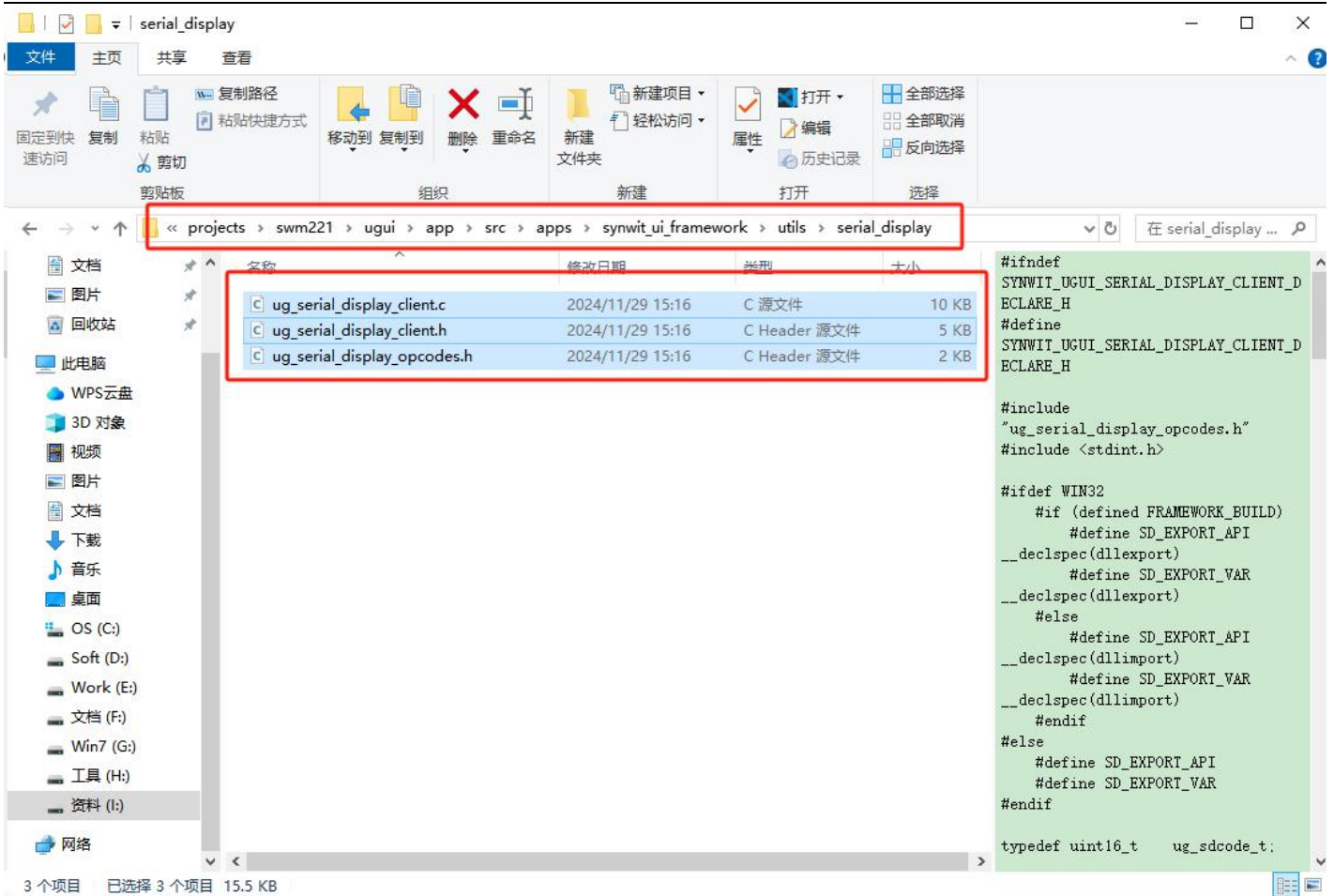
(1) **setter** 控制帧(主控→屏):

字节数	说明	层次
2	帧头(FA 80)	链路层
2	数据长度	
1	MsgId(用户自定义)	数据层
1	指令数	
1	功能码 1	
n	功能码 1 的参数	
1	功能码 2	
n	功能码 2 的参数	
n		
2	CRC16 (可选)	链路层

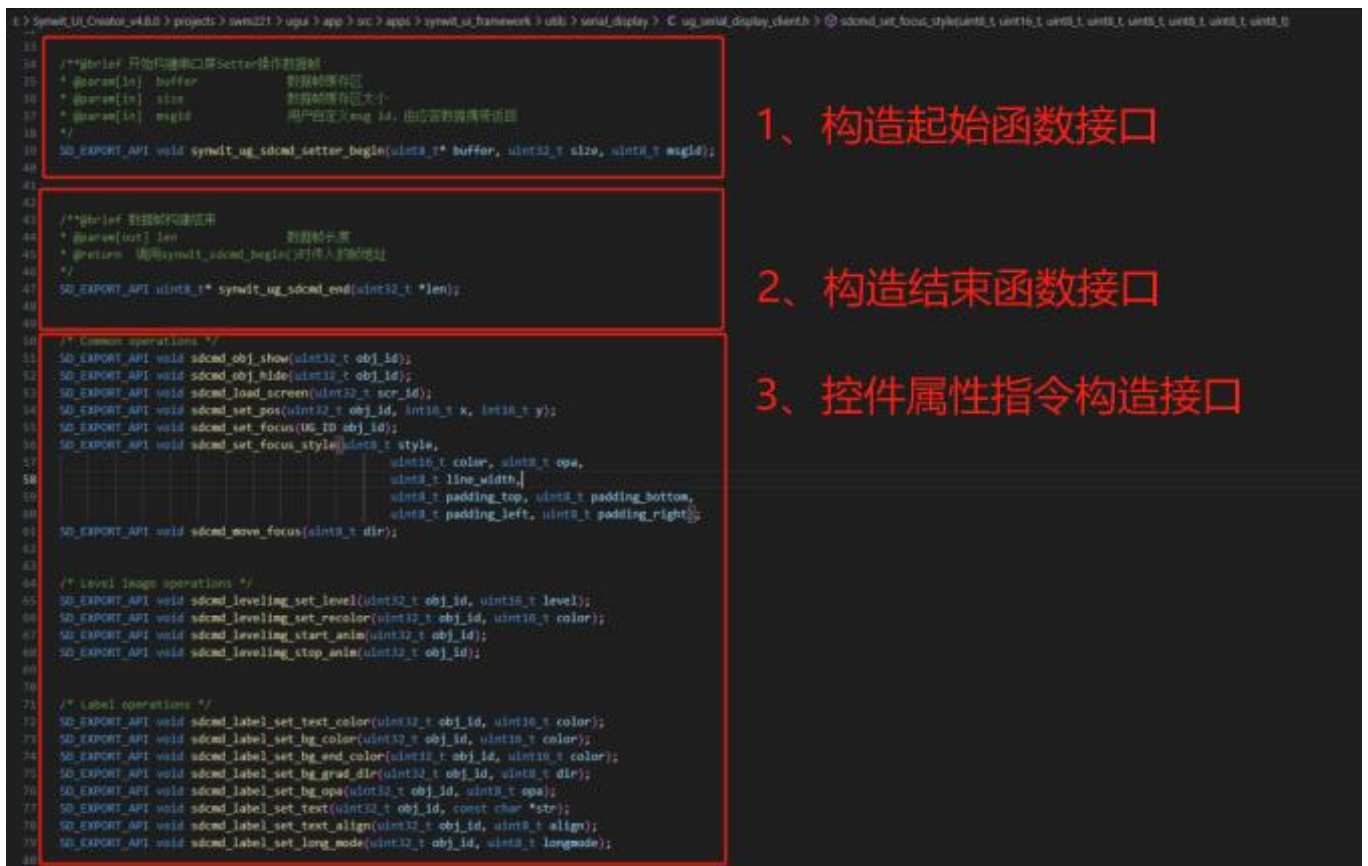
(2) **setter ACK**(屏→主控):

字节数	说明	层次
2	帧头(FB 80)	链路层
2	数据长度	
1	MsgId	数据层
1	执行结果数	
1	执行结果 1	
1	执行结果 2	
1		
1	执行结果 n	
2	CRC16 (可选)	链路层

主控应用程序通过调用的帧数据构造接口来构造包含功能指令的 **setter** 控制帧。使用相关的构造接口函数需从 μGUI 上位机平台的工程文件中将串口功能模块移植到主控系统程序中：以 SWM221 的上位机工程为例，串口功能模块所在路径位置是..\Synwit_UI_Creator_v4.8.0\projects\swm221\ugui\app\src\apps\synwit_ui_framework\utils\serial_display，打开此工程文件目录，将以下程序源码文件拷贝移植到主控程序中。



移植完成后，在 ug_serial_display_client.h 头文件中，可以找到构建控制帧所需要用的接口函数，包括构造控制帧的起始函数接口、结束函数接口以及对应控件各个属性控制的指令构造接口：



1、构造起始函数接口

```
void synwit_ug_sdcmd_setter_begin(uint8_t* buffer, uint32_t size, uint8_t msgid);
```

此接口用于传入开辟的数据帧缓存区，该数据帧缓冲区用于数据指令包的装载，所开辟的缓冲区数组大小由用户自定义大小，可依据包含的控件指令数进行灵活分配，其中 msgid 参数为用户提供了自定义信息标识传入，在构建的数据帧及应答帧中特定数据位体现（参考帧结构定义表格），方便用户进行帧甄别。

2、构造结束函数接口

```
uint8_t* synwit_ug_sdcmd_end(uint32_t *len);
```

此接口在结束构造时使用，用于返回装载完指令控制帧的数据缓存区地址，以及获取数据长度值，获得的帧地址和帧长度即可用于串口进行数据发送。

3、控件指令构造函数接口

针对μGUI 上位机平台提供的控件功能，控件指令构造函数接口涵盖了基本的通用控件属性操作以及不同控件特有的常用属性控制，以此类分，具体函数功能及传参说明如以下：

（1）基本操作

通用操作	帧数据构造接口	功能码	参数									
		0	1	2	3	4	5	6	7	8	9	
显示控件	sdcmd_obj_show	0x01	控件 ID									
隐藏控件	sdcmd_obj_hide	0x02	控件 ID									
界面跳转	sdcmd_load_screen	0x03	界面 ID									
调整位置	sdcmd_set_pos	0x04	控件 ID		x 坐标		y 坐标					
设置焦点控件	sdcmd_set_focus	0x05	控件 ID									
设置聚焦样式	sdcmd_set_focus_style	0x06	样式： 0x00: 包围 边框 0x01: 填充 色	颜色值 (RGB565)		透明度	线条宽度	上侧 留空 空间	下侧 留空 空间	左侧 留空 空间	右侧 留空 空间	
移动焦点	sdcmd_move_focus	0x07	移动 方向： 0x00: 上一 个 0x01: 下一 个 0x02: 上移 0x03: 下移 0x04: 左移 0x05: 右移									

(2) 分级图像

通用操作	帧数据构造接口	功能码	参数			
		0	1	2	3	4
设置分级	sdcmnd_levelimg_set_level	0x10	控件 ID		级号	
重着色 (仅对单色图有效)	sdcmnd_levelimg_set_recolor	0x11	控件 ID		颜色值(RGB565)	
开始自动播放	sdcmnd_levelimg_start_anim	0x12	控件 ID			
停止自动播放	sdcmnd_levelimg_stop_anim	0x13	控件 ID			

(3) 标签

通用操作	帧数据构造接口	功能码	参数					
		0	1	2	3	4	5	6
设置文字颜色	sdcmnd_label_set_text_color	0x20	控件 ID		颜色值(RGB565)			
设置背景颜色	sdcmnd_label_set_bg_color	0x21	控件 ID		颜色值(RGB565)			
设置背景渐变终止色	sdcmnd_label_set_bg_end_color	0x22	控件 ID		颜色值(RGB565)			
设置背景渐变色方向	sdcmnd_label_set_bg_grad_dir	0x23	控件 ID		方向: 0: 无渐变 1: 垂直 2: 水平			
设置背景透明度	sdcmnd_label_set_bg_opa	0x24	控件 ID		透明度			
设置文本	sdcmnd_label_set_text	0x25	控件 ID		字符串长度(不包括结尾 0)		字符串……	0x00
设置文本对齐方式	sdcmnd_label_set_text_align	0x26	控件 ID		对齐方式: 0x01: 左对齐 0x02: 水平居中 0x04: 右对齐 0x08: 顶部对齐 0x10: 垂直居中 0x20: 底部对齐			
设置长文本模式	sdcmnd_label_set_long_mode	0x27	控件 ID		长文本模式: 0: 截断 1: 循环滚动 2: 前后滚动			

(4) 图像按钮

通用操作	帧数据构造接口	功能码	参数					
		0	1	2	3	4	5	6
设置文本	sdcmcmd_imgbtn_set_text	0x30	控件 ID		字符串长度(不包括结尾 0)		字符串……	0x00
设置按钮的开关状态 (仅对开关型按钮有效)	sdcmcmd_imgbtn_set_toggled	0x31	控件 ID		要设置的状态: 0: 松开状态 1: 按压状态			

(5) 按钮

通用操作	帧数据构造接口	功能码	参数					
		0	1	2	3	4	5	6
设置文本	sdcmcmd_btn_set_text	0x40	控件 ID		字符串长度(不包括结尾 0)		字符串……	0x00
设置按钮的开关状态 (仅对开关型按钮有效)	sdcmcmd_btn_set_toggled	0x41	控件 ID		要设置的状态: 0: 松开状态 1: 按压状态			

(6) 图像

通用操作	帧数据构造接口	功能码	参数			
		0	1	2	3	4
设置文字重着色 (仅对单色图有效)	sdcmcmd_img_set_recolor	0x50	控件 ID		颜色值(RGB565)	

(7) 进度条

通用操作	帧数据构造接口	功能码	参数			
		0	1	2	3	4
设置进度值	sdcmcmd_bar_set_value	0x60	控件 ID		进度值	
设置进度最小值	sdcmcmd_bar_set_value_min	0x61	控件 ID		进度最小限值	
设置进度最大值	sdcmcmd_bar_set_value_max	0x62	控件 ID		进度最大限值	

(8) 弧形进度条

通用操作	帧数据构造接口	功能码	参数			
		0	1	2	3	4
设置进度值	sdcmcmd_arc_set_value	0x70	控件 ID		进度值	
设置进度最小值	sdcmcmd_arc_set_value_min	0x71	控件 ID		进度最小限值	
设置进度最大值	sdcmcmd_arc_set_value_max	0x72	控件 ID		进度最大限值	

通过以上接口声明，控制帧的构造步骤主要围绕以下三步：传入数据缓存区开始构建->数据帧添加控件操作命令->结束构建返回数据帧地址。以构建一帧 setter 控制帧为例：

```

01 //创建数据缓存区、数据长度变量
02 uint8_t buf[64];
03 uint32_t data_len;
04 //传入数据缓存区开始构建
05 synwit_ug_sdcmd_setter_begin(buf, sizeof(buf), 1);
06 //以改变分级图像分级为例
07 sdcmd_levelimg_set_level(2, 1);
08 //结束构造，返回帧地址及获取数据长度
09 uint8_t *frame = synwit_ug_sdcmd_end(&data_len);
10 //通过串口发送控制帧
11 uart_send_msg(frame, data_len);

```

(1) 首先，程序中开辟出串口数据缓存数组 buf[64]来存储数据，缓存区大小结合数据实际应用需求设置即可，需声明一个长度变量用于获取数据帧长度。

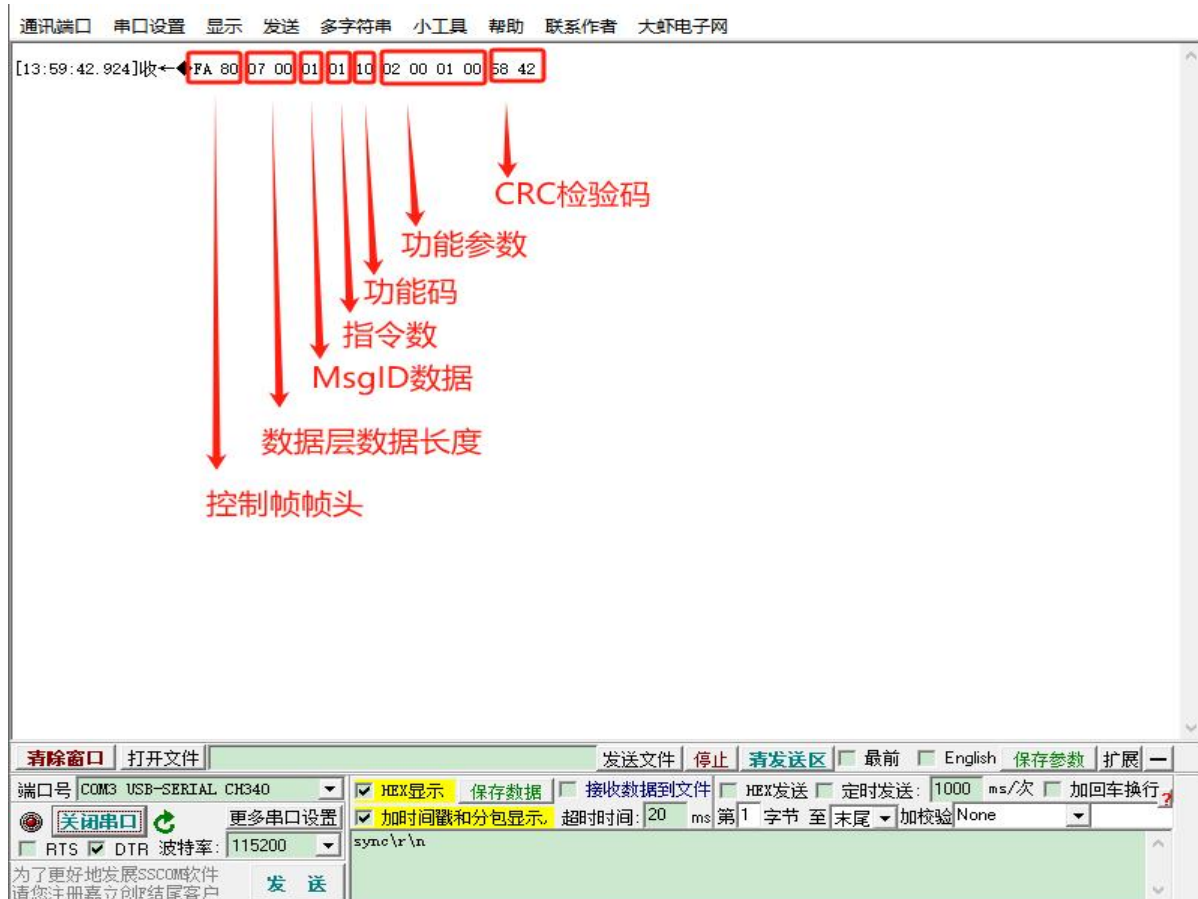
(2) 将缓存区地址传入，设置帧用户自定义数据位 msgid 为 1，启动构建。

(3) 随后添加控件操作命令，这里以可以控制分级图像级数的 sdcmd_levelimg_set_level 接口为例：传入上位机分配的控件 ID 为 2 的分级图像，设置级数为 1，如需一帧中包含多个控件操作指令，可在此步骤中继续调用其他接口进行指令追加。

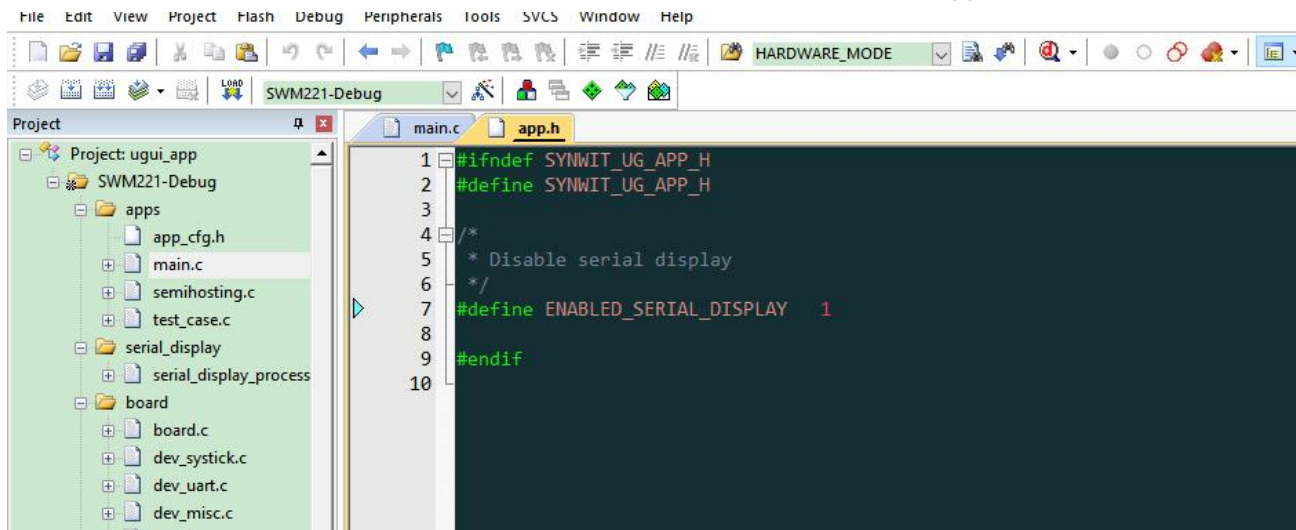
(4) 结束构建，返回控制帧帧地址、获取数据帧长度。

(5) 使用串口发送函数将数据发出。

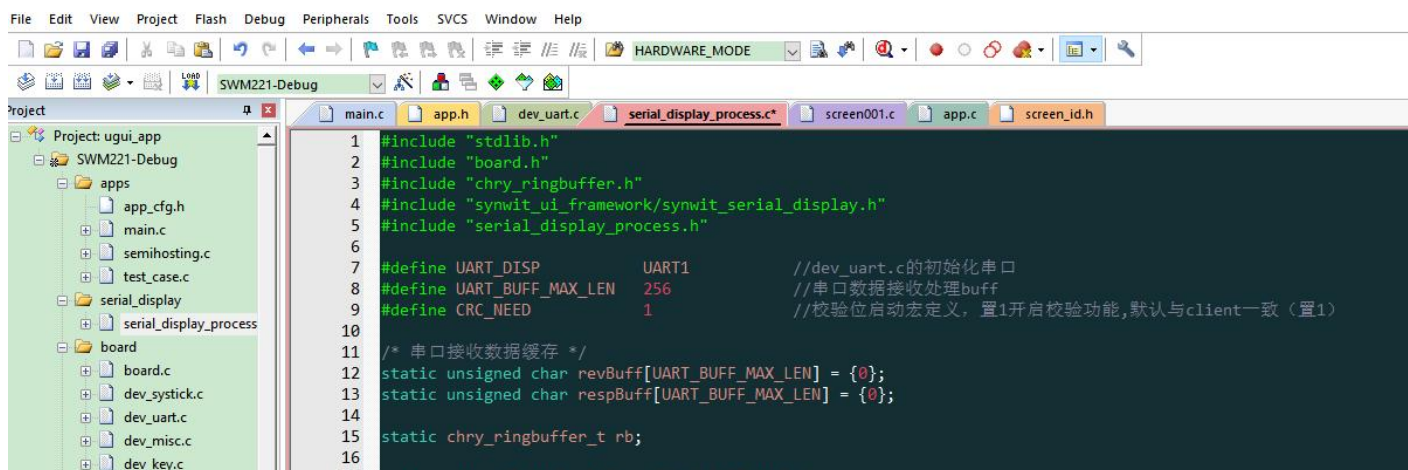
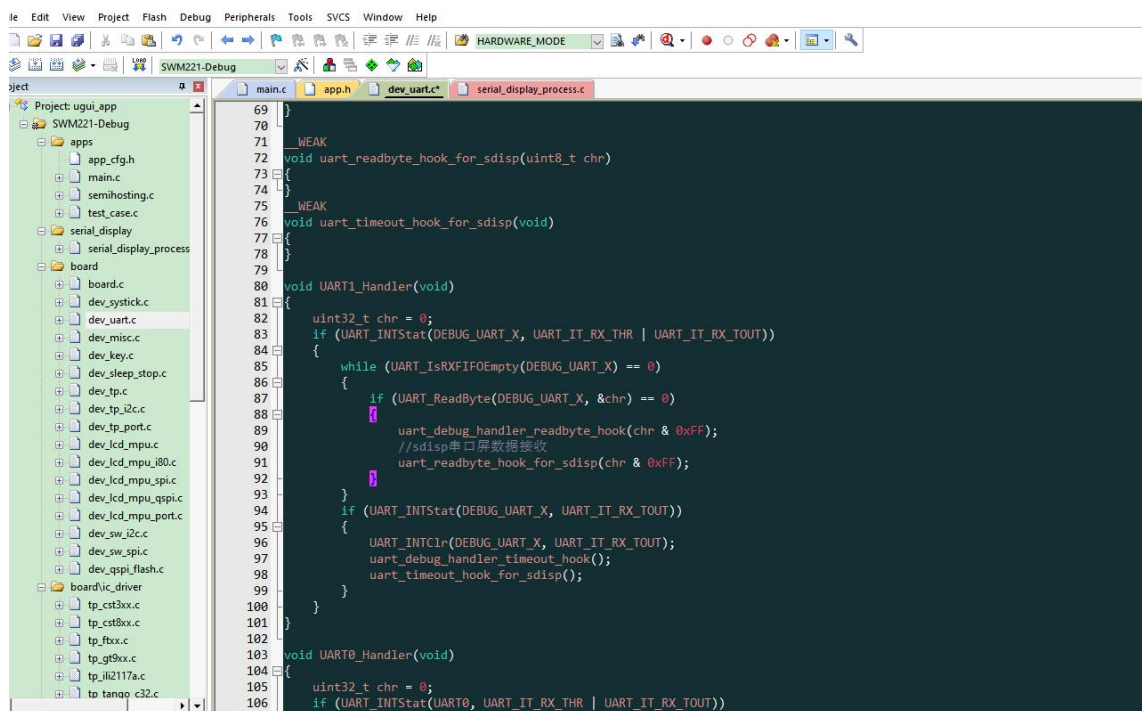
通过执行以上代码步骤，即可构造出一帧控制帧数据并由主控进行输出，将主控的串口通过 USB 线与电脑连接，使用串口助手工具查看数据内容及数据解析如下：



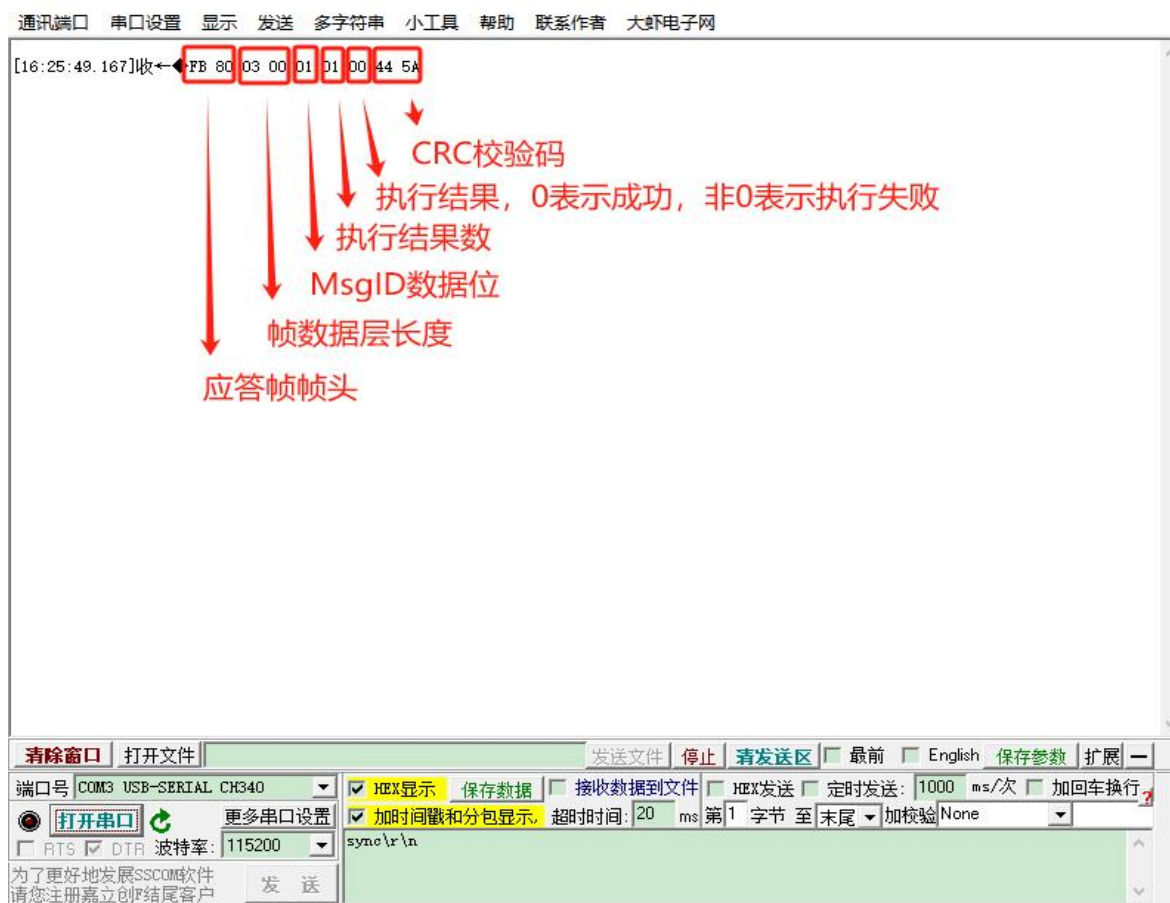
串口屏端需进行上位机工程及素材烧录，上位机默认不开启串口屏解析功能，在 `app.h` 中将串口功能宏置 1 开启：



上位机工程串口屏功能默认使用 UART1 串口号，模式为 8N1，串口相关配置可于 `dev_uart.c` 中进行修改，此默认串口号在 `serial_display_process.c` 中也做发送端口使用，用于应答帧、通知帧及出错帧的发送：



串口屏端配置烧录完毕后，与主控系统进行串口通讯，以解析上述控制帧实例为例，接收到该数据帧后可观测到串口屏屏端显示，控件 ID 为 2 的分级图像跳变为级数 1 对应的图片。使用串口助手工具查看串口屏端 TX 端口发出的应答数据帧，具体解析如下：



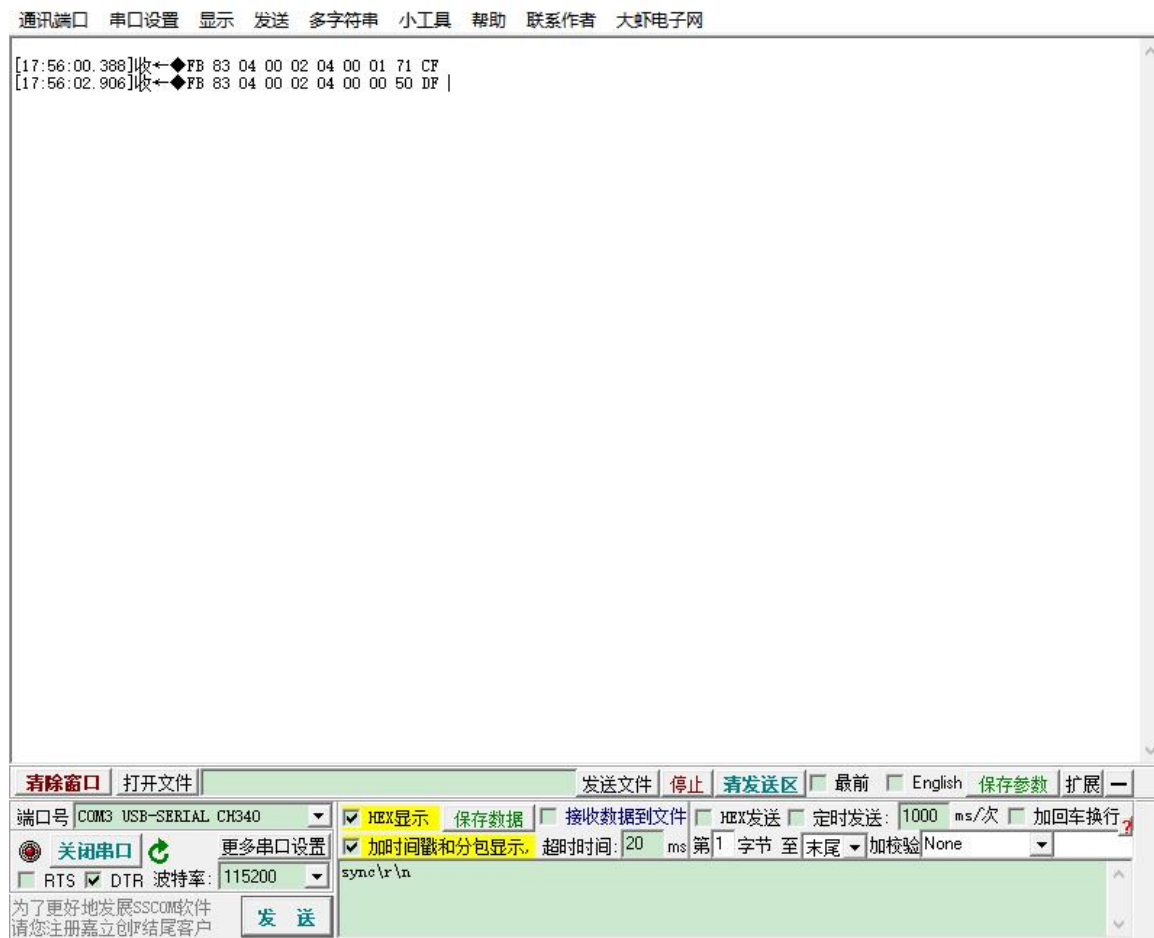
以上是μGUI 上位机平台串口协议控制应答的基本流程。而针对控件状态的获取，平台目前使用通知帧方式，向主控发送控件状态，通知帧的数据结构定义如下：

字节数	说明	层次
2	帧头(FB 83)	链路层
2	数据长度	
n	通知数据	数据层
2	CRC16（可选）	链路层

目前可支持的触发控件类型为按钮及圆弧进度类，具体如下：

通知	通知码	参数				
		1	2	3	4	5
普通型按钮/图像按钮触控通知	0x01	控件 ID		目前固定为 0x05，表示用户点击了某个按钮/图像按钮		
开关型按钮/图像按钮状态改变通知	0x02	控件 ID		切换为按下状态时为 1 切换为弹起状态时为 0		
进度条/圆弧进度条进度改变通知 (仅“触控调节”属性为开启的才会发送此通知)	0x03	控件 ID		调节后的进度值		

以可切换状态的按钮为例，在上位机创建一个可切换图像按钮，控件 ID 为 4，更新素材到串口屏中，在屏上触控图像按钮进行状态切换，此时使用串口助手工具查看通知帧数据如下，可看到通知数据帧与表格结构定义一一对应。



针对诸如检验码错误、对错误控件进行操作等情况，协议使用通信出错帧形式反馈到主控，目前错误码统一使用 0，数据帧具体结构定义如下：

字节数	说明	层次
2	帧头(FB FF)	链路层
1	错误码(目前为 0)	
2	CRC16	

三、 串口协议指令生成工具

为方便对串口协议功能的了解与调试，用户可使用浏览器输入网址 <https://tools.synwit.cn/serial>，进入网站即可使用串口协议指令生成在线工具，工具支持 LVGL 平台和μGUI 平台框架。

使用在线工具，进行 MsgID、控件属性设置等进行修改，点击生成，即可获得对应的数据帧。

华芯微特UI Creator串口协议生成(lvgl)

框架选择: lvgl

消息ID

ID为空时,默认为0

CRC校验

☒ setter ☐ getter

1

标签 -> 文本颜色

23

#862727

新增

删除

✓

生成成功!

生成

生成的数据

FA 80 0A 00 00 01 17 00 0D 00 86 27 27 00

四、 虚拟串口仿真调试

上位机的仿真模拟窗口支持串口屏模拟功能（该功能默认关闭，可从菜单->“工具”->“串口屏模拟串口设置”开启）。开启后，主控系统通过 USB 接入电脑，在接收到主控系统发送的串口指令后，通过解析指令，仿真模拟窗口的控件状态可以做出相应的改变，进而实现串口屏实时模拟调试。



版本记录

版别	发布日期	改版说明
V1.0	2024/10/08	● Preliminary version（初版）。
V2.0	2025/03/06	● 对原有文档说明进行优化，增加μGUI 平台串口协议说明，增加串口协议指令在线生成工具说明。

重要声明 - 请仔细阅读

广东华芯微特集成电路有限公司（简称“华芯微特”）对此文档拥有专属产权。此文档及其中描述的华芯微特产品（下称“产品”）为公司所有。

华芯微特保留随时变更、订正、增强、修改和改良此文档的权利，恕不另行通知。请使用者在下单购买前联系华芯微特获取此文档的最新版本。

华芯微特竭力提供准确可信的资讯，但即便如此，并不推定华芯微特对此文档准确性和可靠性承担责任。买方自行负责对华芯微特产品的选择和使用，华芯微特概不承担与应用协助或买方产品设计相关的任何责任。